UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 07 • **Ejercicio 05** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 5: Sistema de Automatización del Hogar (Smart Home)

Contexto

En este ejercicio exploraremos la coexistencia entre **Clases Abstractas** e **Interfaces de Rol**, aplicando el **Principio de Segregación de Interfaces (ISP)** y profundizando en la distinción fundamental entre **Tipo Aparente** (el tipo de la variable conocido por el compilador para validar los mensajes enviados) y **Tipo Real** (la instancia concreta creada en tiempo de ejecución que define el comportamiento polimórfico).

Enunciado

Se solicita modelar el núcleo de un sistema domótico concentrador (`PanelDeControl`) responsable de coordinar, automatizar y monitorear diversos artefactos inteligentes instalados en un hogar.

Todos los artefactos comparten una base común de hardware modelada mediante la clase abstracta `DispositivoInteligente`. Esta clase define los atributos comunes protegidos (`id: String`, `nombre: String`, `encendido: boolean`), provee métodos concretos para el control elemental de energía (`encender()`, `apagar()`, `isEncendido()`) y declara el método abstracto:

```
public abstract String ejecutarAccionPrincipal();
```

No todos los dispositivos poseen las mismas capacidades de conectividad o medición: algunos requieren enlace inalámbrico para control remoto (`conectarRed(String red)` , `isConectado()`), otros proveen telemetría ambiental (`obtenerLectura(): double` , `getUnidadMedida(): String`), y otros funcionan de forma aislada.

Desafío de Diseño (ISP): Si incorporáramos estas operaciones directamente en la superclase `DispositivoInteligente` , forzaríamos a artefactos simples a heredar métodos ajenos a su función. Diseñe dos interfaces de rol independientes y cohesivas:

1. **`ConectableWiFi`** : Contrato para artefactos que se enlazan a la red inalámbrica del hogar (`conectarRed(String red)` y `boolean isConectado()`).
2. **`SensorMetrico`** : Contrato para artefactos que proveen telemetría ambiental o física (`double obtenerLectura()` y `String getUnidadMedida()`).

Se deben modelar las siguientes especializaciones concretas:

- **`LuzRGB`** : Además del control de energía, gestiona `color: String` e `intensidad: int` (0 a 100). Es una luminaria inteligente que requiere enlace WiFi para operar a distancia. Su acción principal reporta: `"Luz [nombre] iluminando en color [color] con intensidad [intensidad]%"` .
- **`Termostato`** : Modula el clima mediante `temperaturaActual: double` y `temperaturaDeseada: double` . Es un artefacto híbrido: requiere enlace WiFi para control remoto y a la vez actúa como sensor ambiental proveyendo la lectura de temperatura en `"°C"` . Su acción principal reporta: `"Termostato [nombre] regulando temperatura a [temperaturaDeseada] °C"` .
- **`CerraduraElectronica`** : Controla el acceso físico mediante el estado `bloqueada: boolean` , ofreciendo métodos `bloquear()` y `desbloquear()` . Por estrictas razones de seguridad física y aislamiento perimetral, es un artefacto autónomo que no admite enlaces de red ni

emite telemetría métrica. Su acción principal reporta: `"Cerradura [nombre] asegurada: [bloqueada]"`.

El concentrador **`PanelDeControl`** gestiona una colección de dispositivos (`List<DispositivoInteligente>`) y provee las siguientes operaciones:

- `agregarDispositivo(DispositivoInteligente d)`: Registra un dispositivo en el sistema.
- `ejecutarRutinaGeneral(): List<String>`: Recorre polimórficamente todos los dispositivos registrados que se encuentren **encendidos** (`isEncendido() == true`) y recopila los mensajes generados por su `ejecutarAccionPrincipal()`.
- `conectarDispositivosWiFi(String red): int`: Recorre la colección, identifica aquellos dispositivos que posean la capacidad de conectarse a la red inalámbrica, establece su enlace a la red especificada y retorna la cantidad de dispositivos conectados exitosamente.
- `recolectarLecturasMetricas(): List<Double>`: Recorre la colección y consulta la telemetría exclusivamente de aquellos dispositivos que cumplan el contrato de sensor métrico.

Tareas:

1. Diseñe las interfaces de rol `ConectableWiFi` y `SensorMetrico` declarando los métodos atómicos requeridos.
2. Diseñe la clase abstracta `DispositivoInteligente` con su estado protegido, métodos concretos de energía y el método abstracto `ejecutarAccionPrincipal()`.
3. Implemente las clases concretas `LuzRGB`, `Termostato` y `CerraduraElectronica`, deduciendo y aplicando a cada una las interfaces pertinentes según su responsabilidad de dominio.
4. Diseñe la clase `PanelDeControl` administrando la colección polimórfica `List<DispositivoInteligente>` e implemente las rutinas de ejecución general, conexión inalámbrica y recolección métrica.

Restricciones y Tips

- **Segregación de Interfaces (ISP):** No agregue métodos de red o métricas a `DispositivoInteligente`. Una cerradura no debe verse

forzada a implementar métodos de WiFi ni de sensores que no corresponden a su naturaleza.

- **Tipo Aparente vs. Tipo Real:** Si declara `DispositivoInteligente d = new LuzRGB(...)`, analice qué métodos están accesibles a través de la variable `d`. El compilador solo admitirá mensajes definidos en el tipo aparente (`DispositivoInteligente`), mientras que el método ejecutado en tiempo de ejecución responderá dinámicamente al tipo real (`LuzRGB`).
- **Verificación de Capacidades en el Concentrador:** Para operaciones especializadas de red o telemetría en `PanelDeControl`, compruebe si el dispositivo responde a la interfaz correspondiente (`instanceof ConectableWiFi` / `instanceof SensorMetrico`) antes de invocar los métodos del contrato.
- **Estado de Energía:** En `ejecutarRutinaGeneral()`, los dispositivos que se encuentren apagados (`!isEncendido()`) deben ser omitidos de la rutina.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué es un error de diseño consolidar los métodos de `ConectableWiFi` y `SensorMetrico` directamente dentro de la clase abstracta `DispositivoInteligente`? ¿Qué principio SOLID previene este problema?
2. Si una variable está declarada con tipo aparente `DispositivoInteligente disp = new Termostato(...)`, ¿por qué no compila la instrucción `disp.obtenerLectura()` si el termostato efectivamente posee dicho método? ¿Cómo se vincula esto con el rol del compilador y el despacho dinámico?
3. ¿Qué ventajas ofrece agrupar comportamientos heterogéneos mediante interfaces en lugar de intentar forzarlos dentro de un único árbol de herencia rígido?